

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250207>

ToC so far (07.02.) : Lectures

- L1: [MLE Course housekeeping / ML project checklist](#) (16.01.)
- L2: [Tools, setting up development environment](#) (17.01.)
- L3: [Classification](#) (23.01.)
- L4: [Training models](#) (24.01.)
- L5: [Support Vector Machine](#) (30.01.)
- L6: [Decision trees](#) (31.01.)
- L7: [Ensemble Learning and Random Forests](#) (06.02.)

[Seminars](#)

[Notes](#)

Outline : Recap - ML tools

- Python: Functions and classes
- Reading and storing data
- Handling of data - Pandas and numpy frameworks
- Getting training and validation data sets
- ML algorithms in scikit-learn
- Performance evaluation
- Results visualization: Making graphs and diagrams
- Examples

Python : functions, execution flow, arrays...

- Python functions
- Execution flow control
 - If ... Else
 - While Loops
 - For Loops
- Arrays

```
# Download the data
import urllib.request
DOWNLOAD_ROOT = "https://raw.githubusercontent.com/ageron/handson-ml2/master/"
os.makedirs(datapath, exist_ok=True)
for filename in ("oecd_bli_2015.csv", "gdp_per_capita.csv"):
    print("Downloading", filename)
    url = DOWNLOAD_ROOT + "datasets/lifesat/" + filename
    urllib.request.urlretrieve(url, datapath + filename)
```

Python : Classes

Python Classes/Objects

(Python Inheritance)

Reading and storing data

pandas.read_csv

```
import pandas as pd
df= pd.read_csv('C:\\Users\\andreilo\\Desktop\\ToRead\\datasets\\4E.csv',encoding='latin-1')
df.head()
```

DataFrame

.head()
.info()
.describe()
.value_count()
.hist(bins=..., figsize=...)

...

In [9]:

```
%matplotlib inline
import matplotlib.pyplot as plt
housing.hist(bins=50, figsize=(20,15))
save_fig("attribute_histogram_plots")
plt.show()
```

Handling data - pandas and numpy APIs

DataFrame

.iloc()

...

numpy

.dot

...

.linalg

.pinv

.inv

.norm

• ...

```
X = wrkpcs.iloc[:, :4]
```

X

	Length	Width	Height	Diameter
0	450	396	299	111
1	751	155	161	139
2	93	448	293	24
3	443	424	76	165
4	645	521	294	164

```
y = wrkpcs.iloc[:, 4:]
```

y

	Volume
0	5.038841e+07
1	1.629808e+07
2	1.207500e+07
3	1.265016e+07
4	9.258675e+07
...	...

```
wrkpcs.head()
```

	Length	Width	Height	Diameter	Volume
0	450	396	299	111	5.038841e+07
1	751	155	161	139	1.629808e+07
2	93	448	293	24	1.207500e+07
3	443	424	76	165	1.265016e+07
4	645	521	294	164	9.258675e+07

In [4]:

```
X_b = np.c_[np.ones((100, 1)), X] # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

```
X = wrkpcs[['Length', 'Width', 'Height', 'Diameter']]
```

```
y = wrkpcs['Volume']
```

Getting training and validation data sets

from [sklearn.model_selection](#) import [train_test_split](#)
X_train, X_test, y_train, y_test = **train_test_split**(X, y, test_size=0.2, random_state=42)

Also:>

```
import numpy as np

# For illustration only. Sklearn has train_test_split()
def split_train_test(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]

train_set, test_set = split_train_test(wrkpcs, 0.2)
len(train_set)

800

X_train = train_set[['Length', 'Width', 'Height', 'Diameter']]
X_test = test_set[['Length', 'Width', 'Height', 'Diameter']]
y_train = train_set['Volume']
y_test = test_set['Volume']
```


ML algorithms in scikit-learn

Problem specific - regression / classification

alg = LinearRegression() | LinearSVC() | LinearSVR() | SVC() | SVR() |
OneVsRestClassifier() | KNeighborsClassifier() | SGDRegressor() | Lasso() | ElasticNet()
| DecisionTreeClassifier() | DecisionTreeRegressor() | RandomForestClassifier() |
RandomForestRegressor() | ... | Pipeline([("...", alg₁), ("...", alg₂), ..., ("...", alg_n)])

alg.**fit**(X_train, y_train)

alg.**predict**(X_test)

And then comparing the predictions to the y_test to evaluate the performance

Also PolynomialFeatures() + e.g. LinearRegression.

So, same “interface” or polymorphism also allows:

```
def plot_learning_curves(model, X, y):
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=10)
    train_errors, val_errors = [], []
    for m in range(1, len(X_train) + 1):
        model.fit(X_train[:m], y_train[:m])
        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)
        train_errors.append(mean_squared_error(y_train[:m], y_train_predict))
        val_errors.append(mean_squared_error(y_val, y_val_predict))

    plt.plot(np.sqrt(train_errors), "r-+", linewidth=2, label="train")
    plt.plot(np.sqrt(val_errors), "b-", linewidth=3, label="val")
    plt.legend(loc="upper right", fontsize=14) # not shown in the book
    plt.xlabel("Training set size", fontsize=14) # not shown
    plt.ylabel("RMSE", fontsize=14) # not shown
```

```
lin_reg = LinearRegression()
plot_learning_curves(lin_reg, X, y)
plt.axis([0, 80, 0, 3]) # not shown in the book
save_fig("underfitting_learning_curves_plot") # not shown
plt.show() # not shown
```

Performance evaluation

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,  
mean_absolute_percentage_error, r2_score
```

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)  
  
polynomial_regression.fit(X_train, y_train)
```

```
y_predict = polynomial_regression.predict(X_val)
```

```
mean_absolute_error(y_test, y_predict)
```

```
np.float64(1.6001096810214223e-07)
```

```
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(y_train_5, y_train_pred)
```

Results visualization

import matplotlib.pyplot as plt

plt.

plot()

legend()

xlabel()

ylabel()

axis()

grid()

show()

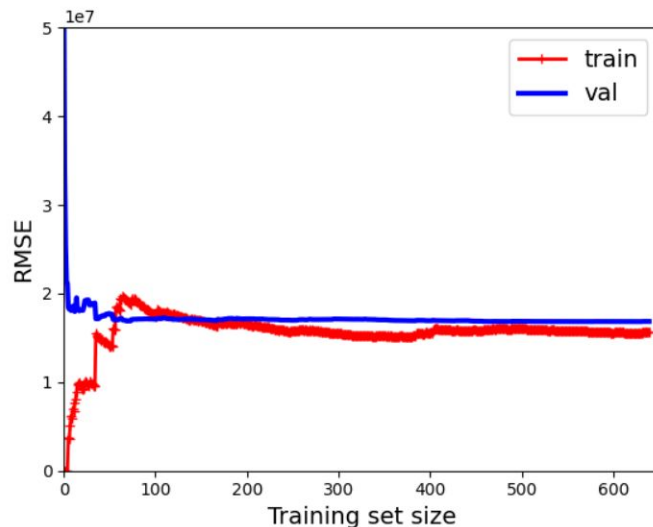
...

```
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

def plot_learning_curves(model, X, y):
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=10)
    train_errors, val_errors = [], []
    for m in range(1, len(X_train) + 1):
        model.fit(X_train[:m], y_train[:m])
        y_train_predict = model.predict(X_train[:m])
        y_val_predict = model.predict(X_val)
        train_errors.append(mean_squared_error(y_train[:m], y_train_predict))
        val_errors.append(mean_squared_error(y_val, y_val_predict))

    plt.plot(np.sqrt(train_errors), "r-+", linewidth=2, label="train")
    plt.plot(np.sqrt(val_errors), "b-", linewidth=3, label="val")
    plt.legend(loc="upper right", fontsize=14) # not shown in the book
    plt.xlabel("Training set size", fontsize=14) # not shown
    plt.ylabel("RMSE", fontsize=14) # not shown

lin_reg = LinearRegression()
plot_learning_curves(lin_reg, X_train, y_train)
plt.axis([0, 650, 0, 50000000]) # not shown in the book
#save_fig("underfitting_Learning_curves_plot") # not shown
plt.show()
```



Now, as a homework

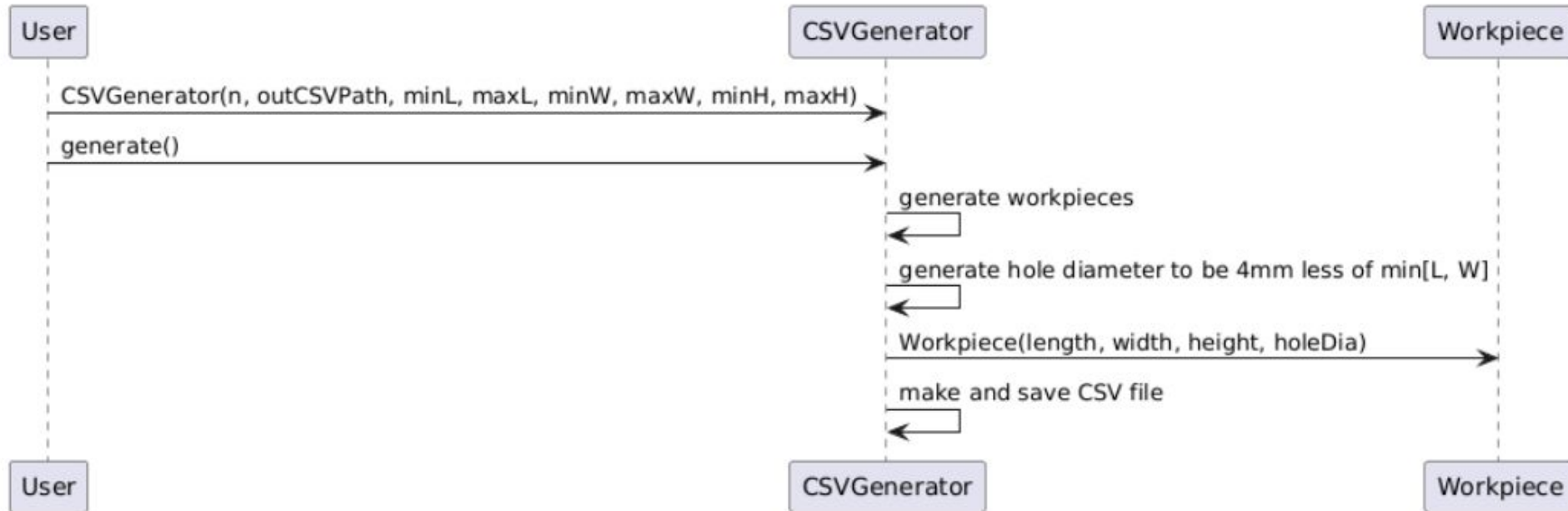
Add new column for a class to data and make the classification. **CSVGenerator** and **Workpiece**

Classes are 'Tall', 'Flat', 'Proportional'.

Tall: Height is significantly (more than 2 times) larger than Length or Width

Flat: Height is significantly smaller (less than 2 times) than Length or Width

Proportional : Length, Width and Height are similar (not exceeding each others values by two times) to each other.



calClass to Workpiece

